

Claude Code for Everyone: A Briefing Doc

Because coding agents broke containment and now your non-programmer friends are building apps

How to Use This Doc

Just want to start? Jump to [Quick Start](#)

Want to understand first? Read the sections in order:

1. [Executive Summary](#) - What is this?
2. [Three Paths](#) - Which tool for you?
3. [Terminal Basics](#) - The scary-looking thing demystified
4. [Getting Started](#) - Step-by-step walkthrough
5. [Security & Privacy](#) - What's safe, what's not

Looking something up? Use Cmd+F to search, or check:

- [Jargon Translation Table](#) - Dev speak → English
- [Key Terms Explained](#) - What does X mean?

Executive Summary (The TL;DR)

What happened: Claude Code (Anthropic's AI coding tool) has "broken containment" into mainstream consciousness. Non-technical people are suddenly realizing they can build actual apps—and you might be one of them.

The big insight: Your Claude Pro subscription (\$20/month) already includes access to an AI that can code on your local Mac - outputs live on your computer, not in some cloud

sandbox. You still get [claude.ai](#) for chat, but now you can also run `claude` in your terminal and build things.

Why this matters: The interface is just... conversation. You describe what you want, Claude builds it.

Quick Start (For the Impatient)

Skip the theory? Here's your 5-minute path to building something:

`bash`

```
# Step 1: Open Terminal
# On Mac: Press Cmd+Space, type "terminal", hit Enter

# Step 2: Install Claude Code (copy-paste this whole line)
curl -fsSL https://claude.ai/install.sh | bash

# Step 3: Create a folder for your project
mkdir ~/my-first-project && cd ~/my-first-project

# Step 4: Launch Claude Code
claude

# Step 5: Tell it what to build (in plain English!)
# Example: "Make me a simple timer with start, stop, and reset buttons"
```

What happens next: Claude creates the files, you open them in your browser, and boom—you built something. Come back to this doc when you want to understand what just happened.

Prefer clicking over typing? Download Claude Desktop from claude.com/download — same coding powers, visual interface.

Three Paths, Choose Your Adventure

Option 1: Claude Desktop (Now With Claude Code!)

- Visual app on your Mac/Windows/Linux
- Familiar chat interface with coding superpowers
- **Current status:** Claude Code is now integrated into the Desktop app
- **Best for:** People who want a visual interface without learning Terminal
- **Bonus:** Can run multiple coding sessions in parallel

Option 2: Claude CLI (What This Doc Covers)

- Terminal-based (that "hacker screen" thing—don't worry, it's friendlier than it looks)
- Type `claude` and get a conversational coding agent
- **Current status:** Live, included with Claude Pro
- **Best for:** Anyone willing to learn basic terminal use
- **Key advantage:** Your code lives on YOUR computer, not in the cloud

Option 3: Continue Using `claude.ai`

- Web interface you know
- Good for chat, artifacts, general use
- Works great with Claude Pro's "don't train on my data" setting
- **Trade-off:** Artifacts are in the cloud (but you can export)

Terminal Basics for Normal Humans

What IS the Terminal?

The honest truth: That black screen with green text from hacker movies? It's just texting your computer instead of clicking buttons. That's it. No magic, no hacking skills required.

Think of it like a restaurant:

- **Clicking around (GUI)** = Looking at a menu with pictures, pointing at what you want
- **Using Terminal (CLI)** = Texting your order directly to the kitchen

Both get you food. The text version is faster once you learn the "menu" (commands). And with Claude Code, you don't even need to memorize the menu—Claude knows it for you.

Why Terminal feels scary (but isn't):

- Movies made it look like hacking
- Mistakes feel permanent (they're usually not—there's usually an undo)
- No pictures or buttons to guide you
- **The reframe:** It's just a very literal conversation. You ask, computer answers.

macOS Terminal location:

- Applications → Utilities → Terminal
- Or: Cmd+Space, type "terminal", hit Enter

What Could Terminal Do Pre-Claude Code?

Your Mac's terminal has always been powerful - here are things you could already do:

File Management (The Basics)

bash

```
pwd                # Print Working Directory - "where am I?"
```

Output: `/Users/yourname` (shows your current location)

bash

```
ls                # List - "show me files here"
```

Output: `Desktop Documents Downloads Pictures` (your folders)

bash

```
cd Documents      # Change Directory - "go into this folder"
pwd               # Check where you are now
```

Output: `/Users/yourname/Documents`

bash

```
mkdir my-project  # Make Directory - "create new folder"
ls                # Check it was created
```

Output: You'll see `my-project` in the list now

Running Programs

bash

```
python script.py  # Run Python code
npm start          # Start a web app
open file.html     # Open file in browser
```

Git (Version Control)

bash

```
git status        # What changed?
git add .          # Stage changes
git commit -m "msg" # Save snapshot
git push           # Upload to GitHub
```

Key insight: The terminal always had these powers. Claude Code just gives you an AI copilot that knows how to use them all.

Security & Privacy Deep Dive

Your Claude Pro Setup

What you opted into:

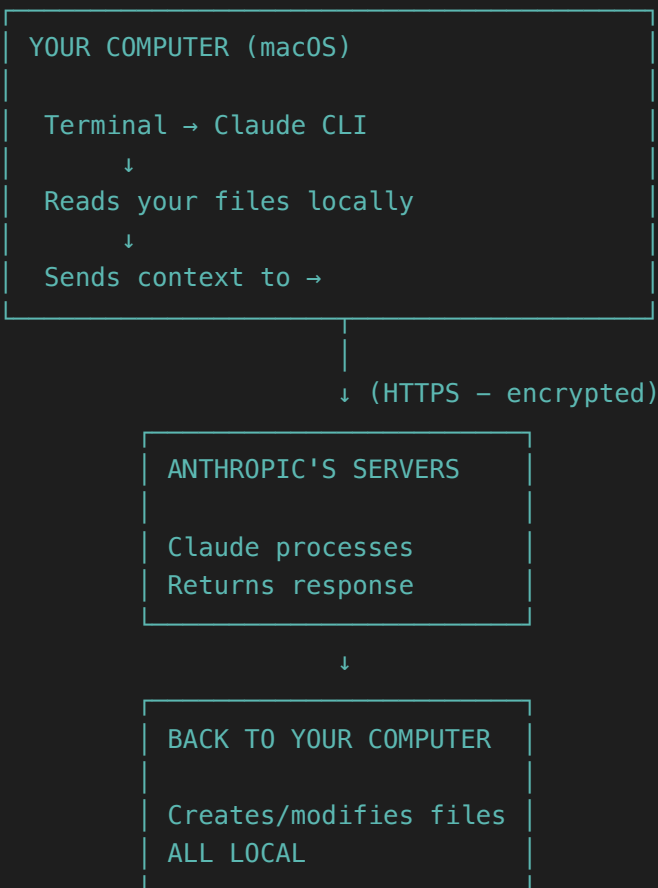
- Claude Pro subscription (\$20/month)
-  **Opted OUT of training** - Your data doesn't improve future models
- This applies to claude.ai AND Claude CLI

What this means:

- 30-day data retention for feedback transcripts (debugging, if things break)
- After 30 days, your conversations get deleted
- Anthropic can't use your code to train new models

Understanding the Data Flow

diagram



The trade-off:

- **Cloud dependency:** Needs internet, sends your code to Anthropic's API
- **Privacy level:** Better than services that train on your data, but not as private as 100% local AI
- **Alternative:** Cursor offers "Privacy Mode" where code never leaves your computer (but different product, different price)

Sandboxing & Safeguards

What is sandboxing?

Think of it like giving someone keys to ONE room in your house, not the whole house:

- They can rearrange furniture in that room (modify project files)
- They can't wander into your bedroom (access personal files elsewhere)

- They definitely can't knock down walls (mess with system files)

The program runs in a restricted environment where it can't:

- Delete your entire hard drive
- Access files outside its designated area
- Install malware
- Mess with system files

Claude Code's approach:

1. **Working directory:** It operates in folders you explicitly navigate to
2. **Permission model:** macOS security prompts when accessing sensitive areas
3. **Git integration:** Version control = undo button for any changes
4. **You're always in the loop:** It shows you what it's doing before executing

Best practices:

```
bash
```

```
# Create a sandbox folder for experiments
mkdir ~/claude-projects
cd ~/claude-projects

# Or use version control
git init my-new-project
cd my-new-project
claude
```

Pro tip: Start projects in a dedicated folder. If things go sideways, you can delete the whole folder. Your real files elsewhere stay safe.

Implementation Strategy Matrix

Quick Comparison: Ways to Build with AI

Feature	Claude CLI	Claude Desktop	claude.ai	Cursor	Bolt/Lovable
Cost	Included w/ Pro	Included w/ Pro	Included w/ Pro	\$20/mo	\$15-20/mo
Where files live	Local	Local	Cloud	Local	Cloud
Internet required	Yes	Yes	Yes	Partial	Yes
GitHub integration	Yes	Yes	Manual	Yes	Built-in
Best for	Terminal users	Visual interface	Quick prototypes	Pro devs	Quick apps
Learning curve	Medium	Low	Low	Medium	Low

Decision Tree: Which Tool When?

diagram

START: What are you building?

- | Quick prototype/demo (< 1 hour)
 - | └ Use: claude.ai Artifacts
 - | └ Single HTML file
 - | └ Share link instantly
 - | └ Limitation: Stuck in cloud
- | Personal project/learning
 - | └ Use: Claude CLI
 - | └ Files on your computer
 - | └ Full control
 - | └ Learn professional workflow
- | Serious app you'll maintain
 - | └ Use: Claude CLI + Git + GitHub
 - | └ Version control from day 1
 - | └ Deploy to GitHub Pages
 - | └ Professional setup
- | Professional development work
 - | └ Consider: Cursor (privacy mode)
 - | └ More control over AI behavior
 - | └ Enterprise features
 - | └ Different pricing model

Getting Started: From Zero to First Project

Prerequisites Checklist

Already have:

- ☒ MacOS computer
- ☒ Claude Pro subscription
- ☒ Opted out of training (Settings on claude.ai)

Need to install:

1. Claude Code (the easiest way - no Node.js needed!)

bash

```
# Open Terminal and paste this one line:  
curl -fsSL https://claude.ai/install.sh | bash
```

That's it! This downloads and installs everything automatically.

2. Git (probably already installed, check with `git --version`)

- If not installed, macOS will prompt you to install it

Alternative installation (if you prefer npm):

- First install Node.js from nodejs.org
- Then: `npm install -g @anthropic-ai/claude-code`

Your First Session (Type Along!)

Step 1: Open Terminal and create a project folder

bash

```
mkdir ~/claude-projects/my-first-app  
cd ~/claude-projects/my-first-app
```

You'll see: Nothing (no news is good news in Terminal!)

Step 2: Start Claude Code

bash

```
claude
```

You'll see: A welcome screen asking what you want to build

Step 3: Tell it what to make (in plain English)

bash

Create a simple timer app with start, stop, and reset buttons

You'll see: Claude thinking, then creating files. It will show you what it's doing.

Step 4: Open your creation

bash

`open index.html`

You'll see: Your browser opens with a working timer! You built that.

Step 5: Want changes? Just ask

bash

Make the buttons bigger and add a dark mode toggle

You'll see: Claude modifies the code. Refresh your browser to see changes.

Understanding the Interface

When you type `claude`, you get:

diagram

```
Welcome to Claude Code
Current directory: ~/claude-projects/...
What would you like to build?
>
```

Key differences from `claude.ai`:

- Can read/write files in current folder
- Can run terminal commands (like `git`, `npm`)

- Can take screenshots (for debugging visual stuff)
 - Can search the web
 - Multi-turn conversations about your codebase
-

Local vs Cloud: The Honest Trade-offs

Local (Claude CLI) Advantages

✓ Ownership:

- Files on your computer = you control them
- No "platform lock-in"
- Works offline (after initial generation)

✓ GitHub workflow:

- Push to GitHub
- Deploy to GitHub Pages (free hosting!)
- Professional version control

✓ Privacy:

- Source code lives locally
- Can disconnect after building

✓ Integration:

- Use your favorite text editor
- Real terminal access
- Professional developer tools

Local Disadvantages

✖ Setup complexity:

- Terminal learning curve
- Initial installation (though it's easier now!)
- Git configuration

✖ No mobile/tablet:

- Must be at your Mac
- Can't tweak from phone (unless GitHub + CodeSandbox)

✖ Internet still needed:

- For Claude AI requests
- For npm packages
- For deploying

Cloud (claude.ai) Advantages

✔ Instant start:

- No installation
- Works anywhere (phone, tablet, any computer)
- Just click "Create Artifact"

✔ Sharing:

- Get a link instantly
- Others can remix
- Built-in collaboration

✔ Lower barrier:

- No terminal needed
- No Git knowledge required
- Visual interface

Cloud Disadvantages

✗ Export complexity:

- Manual copy-paste for complex projects
- Loses some structure
- Need to recreate locally if you want to deploy seriously

✗ Platform dependency:

- Tied to claude.ai availability
- Subject to rate limits
- Can't easily run professional workflows

Exporting from claude.ai to Local

Made something cool in claude.ai and want it on your computer?

The Easy Way (Recommended)

```
bash
```

```
# 1. Start Claude Code locally
claude

# 2. Tell it what you built:
"I made an app in claude.ai that does [describe it].
Can you recreate it locally?"

# Claude will rebuild it with proper file structure.
```

The Manual Way

1. In claude.ai: Click the Artifact → "Copy code"

2. On your Mac: Create a file, paste the code

bash

```
mkdir ~/my-project && cd ~/my-project  
# Use any text editor to create index.html, paste your code  
open index.html # Opens in browser
```

Pro tip: If you think a project might grow, start in Claude Code from the beginning. Easier than exporting later.

Learning Path: Simple → Professional

Start Here (Your First Projects)

Build with just HTML/CSS/JS—no frameworks, no complexity:

- Calculator, timer, to-do list, simple games
- **Why:** Instant results, no setup headaches, learn fundamentals

Level Up (When Ready)

Add version control and multiple files:

bash

```
git init # Track your changes  
git add . && git commit -m "First save" # Save a snapshot
```

Why: Safety net for experiments, professional habit

Go Pro (When You Feel Limited)

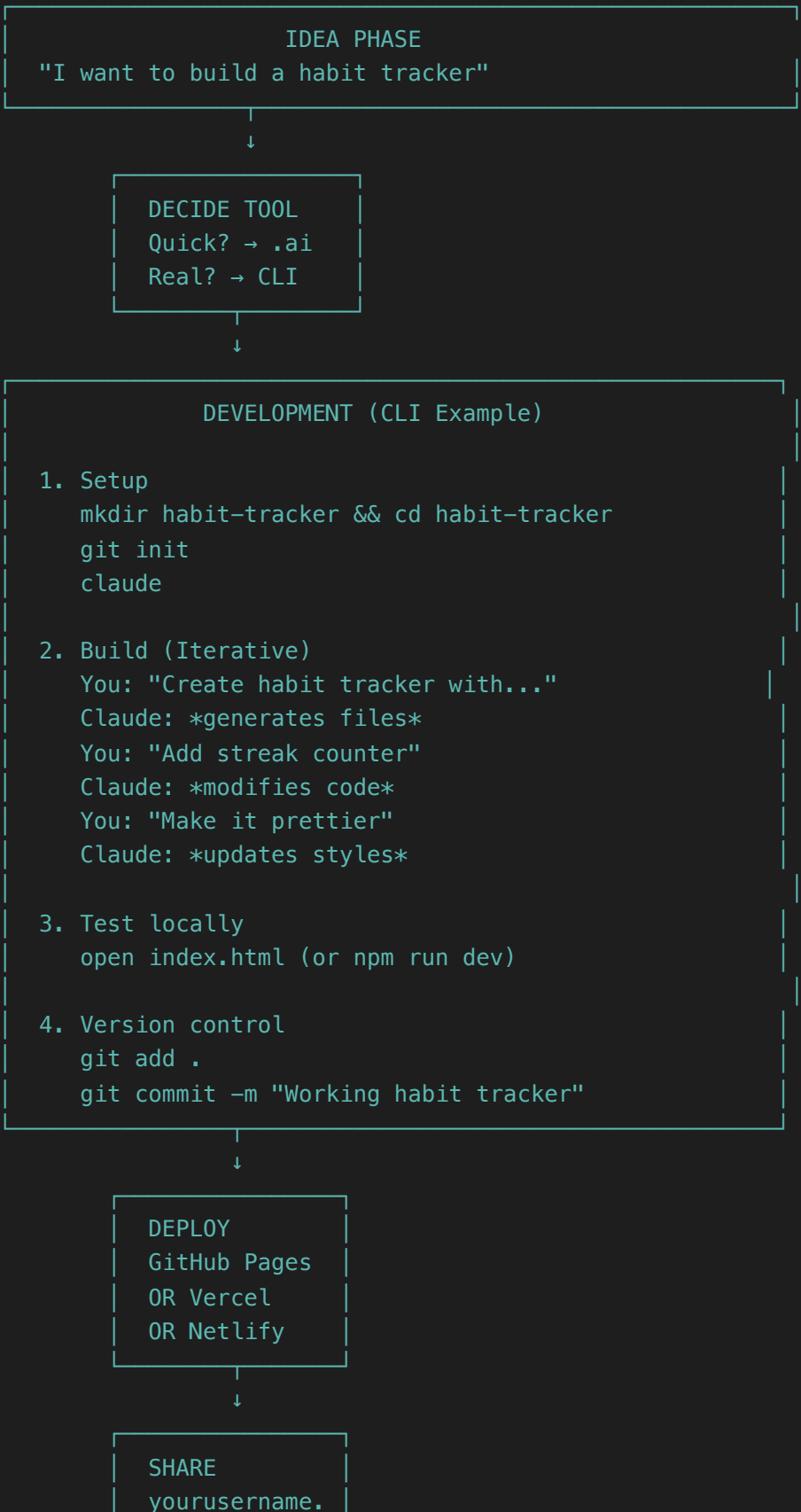
Modern tools solve real problems you'll eventually feel:

- **React:** When your app gets complex and repetitive
- **Vite:** When you need to bundle/optimize code
- **Tailwind:** When CSS becomes unwieldy

Golden rule: Add tools when you feel pain, not before. Claude Code can set up any of these when you're ready—just ask.

Workflow Diagram: The Full Loop

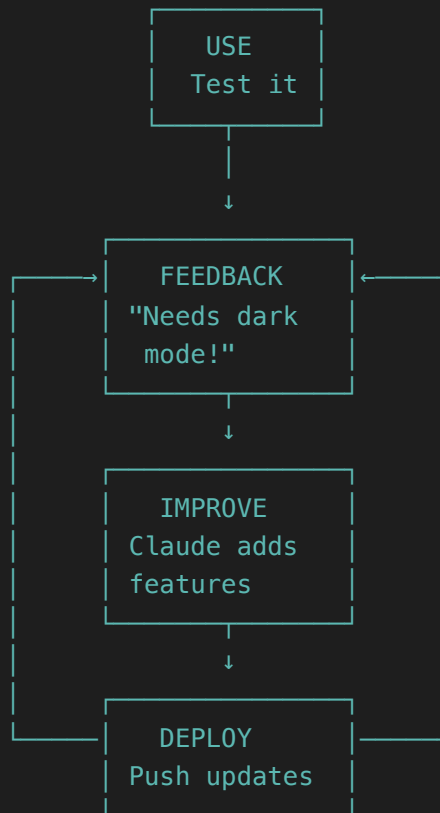
diagram



```
github.io/  
habit-tracker
```

Continuous Iteration Loop

diagram



Conceptual Demystification

Key Terms, Plain English

CLI (Command Line Interface):

- Text-based way to control your computer
- Instead of clicking, you type commands

- Example: `cd Documents` (go to Documents folder)

API (Application Programming Interface):

- How programs talk to each other (like a waiter taking your order to the kitchen)
- Claude Code uses Anthropic's API to access Claude's "brain"
- You don't manage the API key - included in your Pro subscription
- **Don't overthink it:** You use APIs every day without knowing (weather apps, maps, etc.)

Git (Version Control System):

- Like "Track Changes" in Word, but for your whole project folder
- Save snapshots you can return to (like save points in a video game)
- Messed something up? Go back to your last save
- Example: `git commit` = save snapshot

GitHub:

- Website that hosts your Git projects
- Like Dropbox, but designed for code
- Free for public projects

Repository (Repo):

- A project folder with Git tracking
- Lives on GitHub
- Example: `github.com/yourname/habit-tracker`

npm (Node Package Manager):

- Like an App Store, but for code building blocks
- Someone already built a calendar widget? `npm install it` instead of making your own
- Run projects: `npm start` or `npm run dev`

- **Analogy:** Instead of making every ingredient from scratch, you buy pre-made pasta sauce

Localhost:

- Your computer pretending to be a web server
- Address: `localhost:3000` or `localhost:5173`
- Only you can see it (until you deploy)
- **Analogy:** It's like a dress rehearsal. The performance is happening, but the audience (internet) can't see it yet.

Deploy/Deployment:

- Putting your project on the internet
- Going from "only on my computer" to "anyone can visit"
- Services: Vercel, Netlify, GitHub Pages
- **Reframe:** It sounds technical, but these services make it almost as easy as uploading a photo to social media

Build/Build Tool:

- Process that prepares your code for production
- Combines files, optimizes images, etc.
- Tools: Vite, Webpack, Parcel

Environment:

- Where code runs
- Development = on your computer
- Production = on the internet for users

Jargon Translation Table

Dev Speak	Normal Human
"Clone the repo"	"Download the project"
"Push to origin"	"Upload changes to GitHub"
"Merge conflict"	"Two people edited same line"
"npm install"	"Download dependencies"
"Hot reload"	"Auto-refresh when you save"
"Localhost"	"Pretend website on your computer"
"Route"	"Page/screen in the app"
"Component"	"Reusable UI piece"
"Bundle"	"Combine code into one file"
"Deploy"	"Put on the internet"

Strategic Recommendations

For Complete Beginners

Start with **claude.ai** Artifacts:

1. Build 3-5 simple projects in browser
2. Export and learn to run locally
3. Then graduate to Claude CLI

Rationale:

- Lower friction = more learning
- Understand what you're building before worrying about setup
- Tools are means, not end

For "Vibe Coders" (People with Ideas, No CS Degree)**Jump straight to Claude CLI:**

1. One afternoon to learn terminal basics
2. Skip the academic journey, learn by building
3. Non-technical people are doing this successfully—it works!

Best first projects:

- Personal dashboard
- Simple game
- Productivity tool you actually want

Key insight: You don't need to know how every line works. You need to:

- Describe what you want clearly
- Recognize when output is wrong
- Iterate based on testing

Things to Watch Out For**Security Considerations****What Claude Code CAN do:**

- Read any file in current directory

- Write/modify files
- Execute terminal commands
- Install packages (if you confirm)

What it CAN'T do (without you noticing):

- Access files outside your project folder (unless you cd there)
- Run commands without showing you
- Install system-wide software (needs sudo password)

Golden rule:

- Always review code before running
- Use Git so you can undo
- Start in dedicated project folders

Privacy Mindset

Remember:

- Claude Pro = opted out of training 
- But: API calls still processed by Anthropic
- 30-day retention for debugging

For sensitive projects:

- Consider: Running on separate machine
- Or: Use Cursor with Privacy Mode (different product)
- Or: Local-only AI models (more technical, less capable)

Cost Management

Claude Pro (\$20/mo) includes:

- ~45 messages every 5 hours (for typical conversations)

- Weekly usage caps (added in 2025 to ensure fair access)
- 200,000-token context window (that's a LOT of code)
- Priority access during busy times
- Resets regularly, so use it!

Watch out for:

- Long-running tasks can use multiple messages from your quota
- Complex projects might hit limits faster
- Solution: Upgrade to Max (\$100/mo for 5x usage, \$200/mo for 20x) if needed

The "Magic Black Box" Problem**Don't fall into trap of:**

- Accepting all code without reading
- Not understanding basic structure
- Becoming dependent without fundamentals

Better approach:

- Ask Claude to explain major sections
- Read the comments it writes
- Modify small things yourself to learn

Quick Deploy Guide: GitHub Pages**Why GitHub Pages:**

- Free
- Easy
- No server needed

- Custom domain support
- Perfect for static sites

5-Minute Deploy:

bash

```
# 1. In your project
git init
git add .
git commit -m "Initial commit"

# 2. Create repo on GitHub (via website)
# Then link it:
git remote add origin https://github.com/yourusername/projectname.git
git push -u origin main

# 3. Enable GitHub Pages
# Go to repo Settings → Pages
# Source: main branch → / (root) → Save

# 4. Visit your site!
# https://yourusername.github.io/projectname
```

For React/Vite projects:

bash

```
# Add to vite.config.js:
base: '/projectname/'

# Build:
npm run build

# Deploy dist/ folder
# (Use gh-pages package or manual)
```

Other Tools Worth Knowing About

Tool	What It Is	When to Consider
VS Code + GitHub Copilot	Code editor with AI autocomplete	You want traditional dev experience
Cursor	VS Code fork with deeper AI integration	Privacy-focused, or power users
Lovable/Bolt/Replit	Browser-based AI coding	Zero setup, works on any device
claude.ai Artifacts	Quick prototypes in browser	Fast experiments before going local

Bottom line: Start with Claude Code. Explore others later if you hit specific needs.

Next Steps: Your Action Plan

This Week

- ❑ Open Terminal (Cmd+Space, type "terminal", hit Enter)
- ❑ Try basic commands: `pwd` (where am I?), `ls` (what's here?)
- ❑ Install Claude Code: `curl -fsSL https://claude.ai/install.sh | bash`
- ❑ Test it worked: `claude --version`
- ❑ If something feels broken: `claude doctor` (auto-diagnoses issues)

Next Week

- ❑ Create `~/claude-projects` folder

- ☐ `cd ~/claude-projects`
- ☐ `claude` - have first conversation
- ☐ Build tiny project (calculator, timer)
- ☐ Open result in browser

Next Month

- ☐ Learn basic Git (`git init` , `add` , `commit`)
- ☐ Create GitHub account
- ☐ Push first project to GitHub
- ☐ Deploy to GitHub Pages
- ☐ Share link with friend

Next Quarter

- ☐ Build project you actually want to use
- ☐ Learn React basics (when vanilla JS feels limiting)
- ☐ Contribute to open source (even just documentation!)
- ☐ Help someone else get started

Final Thoughts: The New Normal

What's actually changed:

- The barriers to building software are collapsing
- "I'm not technical" matters less than "I can describe what I want clearly"
- The skill shifted from memorizing syntax to clear thinking and iteration

What you still need to learn:

- How to describe what you want (prompt craft)
- How to recognize when output is wrong
- How to test and iterate
- Basic concepts (this doc helps!)

Your advantage as a beginner:

- No bad habits to unlearn
- Fresh perspective on how tools should work
- You'll ask "why is this hard?" when veterans just accept it

What this enables:

- Build tools for your specific weird needs
- Prototype ideas in hours, not weeks
- Actually understand what software can do for you

So: Open that terminal. Type `claude`. Build something small. See what happens.

The files really do live on your computer. That's the point.

Resources

Official:

- code.claude.com/docs - Claude Code documentation
- claude.com/download - Desktop app download
- github.com/anthropics/claude-code - Official repo

If Something Breaks:

- Run `claude doctor` - auto-diagnoses most issues
- [Claude Help Center](#) - Official support

Learning (Free):

- Harvard CS50 - Best intro to computer science
- The Odin Project - Web development fundamentals
- Search "vibe coding" on YouTube - See real examples

Communities:

- r/ClaudeAI on Reddit
- Search #ClaudeCode on X/Twitter

Last updated: January 2026. If something's outdated, the beauty of local files is you can just... edit it.